

1. Rotation 값 복원과 1round Permutation 구조 추정

둘 모두를 동시에 복원해야 일치하는 테스트 벡터가 나타난다. permutation을 구성하는 연산 종류가 4가지이므로, 추정해야 하는 경우의 수는 24가지가 되며, Rotation값을 복원하기 위해 추가로 7비트씩 4개인 28비트를 추정해야한다. 임의로 코드를 수정하는 것이 제한되고 있으므로 출력을 파이프라인을 통해 외부로 저장하여 확인하는 방식으로 조사하려고 한다.

먼저 라운드를 구성하는 함수의 특징은 256비트 단위로 이루어지는 것 같은 연산이 shuffle을 제외하곤 64비트 단위로 이루어진다는 것이다 따라서, Rot의 적용 범위가 되는 워드 하나를 대상으로 추적하여 순서를 무작위로 부여하고 출력 결과를 얻는다.

당연히 MISMATCH가 발생하므로, Permutation_1round는 1회만 실행되고 멈춘다.

입력 데이터의 첫 워드에 적용될 rot[0]값은 shuffle에 의해 가장 마지막 워드에 작용하게 된다. rot[1]값은 그 이전 워드에, rot[2]값은 두 번째 워드에, rot[3]값은 첫 번째 워드에 작용하게 된다.

우리가 테스트할 값은 입력 내부에서 다음과 같다.

```
#1
input
55dedbfa2efbd349 56ff5e7f23b80819 75f6ffe773d535a3 ec7fe7d57fbb91d4
output
295535ca90401af3 87ba7a00f67bb26d 874f090661a9e8b8 9f60fb2910d264d0
```

switch case문으로 24 by 4의 2차원 배열로 구현하여 순서에 대한 전수조사를 진행하였고 출력 시 라벨을 사용하기 위해 sprintf를 사용하여 실행 결과에 라벨과 실행 결과가 출력되도록 했다. 여기서 순서 번호 1은 add, 2는 rot, 3은 shuffle, 4는 xor을 의미한다.

```
void permute_one_round(state256_t *state,
                        const unsigned int rot[4],
                        const uint8_t shuffle_map[32],
                        const uint64_t constants2[4],
                        const uint64_t constants1[4]) {
    (void)rot;
    const int orders[24][4] = {
        {1, 2, 3, 4}, {1, 2, 4, 3}, {1, 3, 2, 4},
        {1, 3, 4, 2}, {1, 4, 2, 3}, {1, 4, 3, 2},
        {2, 1, 3, 4}, {2, 1, 4, 3}, {2, 3, 1, 4},
        {2, 3, 4, 1}, {2, 4, 1, 3}, {2, 4, 3, 1},
        {3, 1, 2, 4}, {3, 1, 4, 2}, {3, 2, 1, 4},
        {3, 2, 4, 1}, {3, 4, 1, 2}, {3, 4, 2, 1},
        {4, 1, 2, 3}, {4, 1, 3, 2}, {4, 2, 1, 3},
        {4, 2, 3, 1}, {4, 3, 1, 2}, {4, 3, 2, 1}
    };

    for (int order_index = 0; order_index < 24; order_index++) {
        for (unsigned int candidate_rot = 1;
             candidate_rot <= 63;
             candidate_rot++) {
```

```

state256_t candidate = *state;
unsigned int trial_rot[4] = {
    candidate_rot, candidate_rot,
    candidate_rot, candidate_rot
};
char label[32];

for (int step = 0; step < 4; step++) {
    switch (orders[order_index][step]) {
        case 1:
            add_constants_64wise(&candidate, constants1);
            break;
        case 2:
            rotate_words_left_64wise(&candidate, trial_rot);
            break;
        case 3:
            shuffle_bytes_256(&candidate, shuffle_map);
            break;
        case 4:
            xor_constants_256wise(&candidate, constants2);
            break;
    }
}

snprintf(label, sizeof(label), "%d%d%d%d rot=%u",
        orders[order_index][0], orders[order_index][1],
        orders[order_index][2], orders[order_index][3],
        candidate_rot);
print_state256(label, &candidate);
}
}
}

```

어차피 rot값에는 서로 다른 정수가 들어가기 때문에 임의의 rot값을 추측한 값으로 4개 다 채운다. 그렇게 하면 하나의 출력 결과를 통해 모든 rot값을 조사하는 것이 가능하다.

이제 run_contest.sh | cat > rot.txt로 결과를 저장하고, 해당 결과를 grep으로 조사한다.

결과는 다음과 같이 나타난다.

```

=== Test 1: one round I/O ===
1234 rot=1 = 311f6012ab5b9ea6 f6341a75c598d34f 709034d0b5904f91 3e0b6f8e7e6c624f
1234 rot=2 = 4af4b6fae296bc13 c2b8a0a92d4d0703 a53186d2fc8323fa c06479332c5c9411
...
4321 rot=63 = 381b0fb7033a6824 46bf749cc051ab9b de9723a3d2df6079 9b392caf84e67570
one-round testvector verification: MISMATCH
=== Test 2: 20 rounds ===
20-round testvector verification: MISMATCH
=== Test 3: timing of 20-round permutation ===
benchmark final state = 0123456789abcdef fedcba9876543210 0f1e2d3c4b5a6978 8877665544332211

```

```
iterations          = 1000000
total elapsed time   = 0.000001 sec
average per 20rounds = 0.000001 us
```

grep 실행 결과:

```
>>> grep "9f60fb2910d264d0" rot.txt
2431 rot=43 = 07e56ecf14b7e018 661e3857dcc085aa 117cb22c3b3c182d 9f60fb2910d264d0
>>> grep "874f090661a9e8b8" rot.txt
2431 rot=7 = 8324e0b140f51cb5 49b49584f328b778 874f090661a9e8b8 fc34309f8a3c159f
>>> grep "87ba7a00f67bb26d" rot.txt
2431 rot=29 = 0f171ff0bc6d1925 87ba7a00f67bb26d 636ece7bef834151 a76aded44fc2a839
>>> grep "295535ca90401af3" rot.txt
2431 rot=14 = 295535ca90401af3 06b7a3de370531bc 73180764823ad920 9fe50827211188e8
```

따라서 rot 배열의 값은 {43,7,29,14}가 되며, 암호화 순서도 rot -> xor -> shuffle -> add가 됨을 확인할 수 있다.

수정 가능한 영역을 다음과 같이 수정하였다.

```
void permute_one_round(state256_t *state,
                        const unsigned int rot[4],
                        const uint8_t shuffle_map[32],
                        const uint64_t constants2[4],
                        const uint64_t constants1[4]) {
    rotate_words_left_64wise(state, rot);
    xor_constants_256wise(state, constants2);
    shuffle_bytes_256(state, shuffle_map);
    add_constants_64wise(state, constants1);
}

/* -----
 * 2) 20-round permutation
 * uses the same constants1/constants2 for all rounds
 * ----- */
void permute_20rounds(state256_t *state,
                      const unsigned int rot[4],
                      const uint8_t shuffle_map[32],
                      const uint64_t constants1[4],
                      const uint64_t constants2[4]) {
    for (int r = 0; r < 20; r++) {
        permute_one_round(state, rot, shuffle_map, constants2, constants1);
        // TODO: Complete this implementation.
    }
}

/* -----
 * 3) Main: test + timing
 * ----- */
int main(void) {
```

```

/* one-round test parameters */
// TODO: Set rot to proper values.
const unsigned int rot[4] = { 43, 7, 29, 14 };

```

이제 실행 결과는 다음과 같다.

```

=== Test 1: one round I/O ===
one-round testvector verification: OK (1000 pairs checked)

=== Test 2: 20 rounds ===
20-round testvector verification: OK

=== Test 3: timing of 20-round permutation ===
benchmark final state = 407b6c00d4644ffb 7b5eeeb7bbbfd53 787627ff592edbdb 942319215bb84f88
iterations           = 1000000
total elapsed time    = 0.431251 sec
average per 20rounds = 0.431251 us

```

2. 완성한 permutation의 최적화 구현

수정한 20round는 다음과 같이 구성된다.

```

void permute_20rounds(state256_t *state,
                    const unsigned int rot[4],
                    const uint8_t shuffle_map[32],
                    const uint64_t constants1[4],
                    const uint64_t constants2[4]) {
    for (int r = 0; r < 20; r++) {
        __m256i x = _mm256_loadu_si256((const __m256i_u *)state->w);
        const __m128i r32 = _mm_loadu_si128((const __m128i_u *)rot);
        const __m256i r0 = _mm256_cvtepu32_epi64(r32);
        const __m256i rinv0 =
            _mm256_sub_epi64(_mm256_set1_epi64x(64), r0);
        const __m256i r1 =
            _mm256_permute4x64_epi64(r0, _MM_SHUFFLE(0, 1, 2, 3));
        const __m256i rinv1 =
            _mm256_permute4x64_epi64(rinv0, _MM_SHUFFLE(0, 1, 2, 3));
        const __m256i vc2 = _mm256_set_epi64x(
            (long long)constants2[3], (long long)constants2[2],
            (long long)constants2[1], (long long)constants2[0]);
        const __m256i vc1 = _mm256_set_epi64x(
            (long long)constants1[3], (long long)constants1[2],
            (long long)constants1[1], (long long)constants1[0]);
        const __m256i vc2_rev = _mm256_set_epi64x(
            (long long)constants2[0], (long long)constants2[1],
            (long long)constants2[2], (long long)constants2[3]);
        const __m256i vc1_rev = _mm256_set_epi64x(
            (long long)constants1[0], (long long)constants1[1],
            (long long)constants1[2], (long long)constants1[3]);
        const __m256i byte_reverse_mask = _mm256_setr_epi8(

```

```

7, 6, 5, 4, 3, 2, 1, 0,
15, 14, 13, 12, 11, 10, 9, 8,
7, 6, 5, 4, 3, 2, 1, 0,
15, 14, 13, 12, 11, 10, 9, 8);

```

```

(void)shuffle_map;

```

```

#pragma GCC unroll 10
for (int i = 0; i < 10; i++) {
    x = _mm256_or_si256(_mm256_sllv_epi64(x, r0),
                        _mm256_srlv_epi64(x, rinv0));
    x = _mm256_xor_si256(x, vc2);
    x = _mm256_shuffle_epi8(x, byte_reverse_mask);
    x = _mm256_add_epi64(x, vc1_rev);

    x = _mm256_or_si256(_mm256_sllv_epi64(x, r1),
                        _mm256_srlv_epi64(x, rinv1));
    x = _mm256_xor_si256(x, vc2_rev);
    x = _mm256_shuffle_epi8(x, byte_reverse_mask);
    x = _mm256_add_epi64(x, vc1);
}

_mm256_storeu_si256((__m256i_u *)state->w, x);
break;
}
}

```

2.1 256-bit SIMD를 이용한 4개 word 병렬 처리

네 word를 한 번에 계산하기 위해 `immintrin.h`와 `_m256i` 자료형을 사용하였다.

`state`의 네 64-bit word는 다음 명령으로 하나의 AVX2 register에 적재하였으며, xor과 add에도 다음 명령어를 사용하였다.

```

_mm256_loadu_si256
_mm256_xor_si256
_mm256_add_epi64

```

이를 통해 네 번의 scalar 연산 대신 한 번의 256-bit SIMD 명령으로 네 word를 처리하였다. 계산이 끝난 `state`는 `_mm256_storeu_si256`으로 한 번에 저장하였다.

2.2 rot의 SIMD 처리

rot {43, 7, 29, 14}를 word마다 각각 적용해야 하지만 AVX2에는 이에 직접 대응하는 rotate 명령이 없다. 따라서 다음 식을 사용해 회전을 구현하였다.

$ROTL(x, r) = (x \ll r) \text{ OR } (x \gg (64-r))$

`rotl64x4_avx2` 함수에서 `_mm256_sllv_epi64`와 `_mm256_srlv_epi64`로 각 word를 서로 다른 값만큼 이동한 뒤, `_mm256_or_si256`으로 두 결과를 합쳤다.

```

left  = _mm256_sllv_epi64(x, left_count)
right = _mm256_srlv_epi64(x, right_count)
result = _mm256_or_si256(left, right)

```

회전량은 보조 함수 안에 다시 고정하지 않고 문제에서 수정이 허용된 rot 배열을 그대로 전달받아 사용하였다. 먼저 `_mm_loadu_si128`로 `rot[0]`부터 `rot[3]`까지 읽고, `_mm256_cvtepu32_epi64`로 네 개의 64-bit 값으로 변환하였

다. 오른쪽 shift에 사용할 값은 64에서 각 회전량을 빼서 계산하였다. 20-round 함수에서는 이 변환을 라운드 반복문 밖에서 한 번만 수행한다. 따라서 회전량은 지정된 rot 배열에서 가져오면서도 같은 값을 매 라운드마다 다시 구성하지 않는다.

2.3 고정 byte shuffle의 SIMD 변환

suffle에서 제공하는 32-byte 전체 역순을 다음 두 동작으로 나누어 구현하였다.

1. 각 64-bit word 내부의 8개 byte 순서를 역순으로 변경
2. 네 64-bit word의 순서를 [0,1,2,3]에서 [3,2,1,0]으로 변경

bswap64x4_avx2 함수에서는 고정 mask와 _mm256_shuffle_epi8을 사용해 네 word 내부의 byte 순서를 동시에 뒤집었다. 이어서 _mm256_permute4x64_epi64로 word 순서를 역순으로 변경하였다. 이 구현으로 두 번의 memcpy, 32회 byte 반복문 및 실행 중 shuffle_map 접근을 제거하였다.

2.4 두 라운드 합성을 이용한 word 순서 변경 제거

매 라운드마다 word 순서를 직접 뒤집는 비용을 줄이기 위해 두 라운드를 하나의 단위로 묶었다. 전체 byte 역순을 두 번 적용하면 word 순서가 원래대로 돌아오는 성질을 이용하였다.

$\text{shuffle}(\text{shuffle}([w_0, w_1, w_2, w_3])) = [w_0, w_1, w_2, w_3]$

permute_20rounds_avx2_2round 함수에서 두 라운드를 하나의 단위로 합성하였다. 반복문의 핵심 부분에서는 word 내부 byte 순서를 바꾸는 _mm256_shuffle_epi8만 실행하고, 매 라운드마다 별도의 word 순서 반전 명령을 실행하지 않도록 구성하였다. 첫 번째 라운드 후에는 register의 word가 논리적인 state와 반대 순서이므로, 두 번째 라운드에 사용하는 지역 변수들을 다음과 같이 반대 순서로 배치하였다.

r1, rinvs1 : 반대 word 순서에 맞춘 회전량

vc2_rev : 반대 word 순서의 constants2

vc1_rev : 반대 word 순서의 constants1

두 번째 라운드가 끝나면 word 순서 반전이 상쇄된다. 이 구조를 10회 적용해 20라운드를 처리하였다.

2.5 20-round loop 전개

고정된 반복문의 제어 비용을 줄이기 위해 다음 지시문을 사용하였다.

```
#pragma GCC unroll 10
```

반복 횟수가 10회로 고정되어 있으므로 GCC가 loop를 전개하도록 하였다. 이를 통해 counter 증가와 분기 비용을 줄이고, 컴파일러가 연속 라운드의 명령 배치를 함께 최적화할 수 있도록 하였다.

또한 state를 20라운드가 끝날 때까지 하나의 __m256i register에 유지하였다.

함수 시작과 종료 시점에만 load와 store를 수행해 라운드 사이의 메모리 접근을 제거하였다.

컴파일 시 AVX2 intrinsic을 활성화하기 위해 run_contest.sh를 다음과 같이 최적화 플래그를 추가하여 수정하였다.

```
#!/usr/bin/env bash
set -euo pipefail
cd "$(dirname "$0")"
gcc -O3 -Wall -Wextra -mavx2 -o contest contest.c
./contest
```

실행 결과는 다음과 같다.

```
>>> ./run_contest.sh
=== Test 1: one round I/O ===
one-round testvector verification: OK (1000 pairs checked)
```

=== Test 2: 20 rounds ===

20-round testvector verification: OK

=== Test 3: timing of 20-round permutation ===

benchmark final state = 407b6c00d4644ffb 7b5eeeeb7bbbfd53 787627ff592edbdb 942319215bb84f88

iterations = 1000000

total elapsed time = 0.024907 sec

average per 20rounds = 0.024907 us